HAtt: Hybrid Remote Attestation for the Internet of Things With High Availability

Muhammad Naveed Aman, Mohamed Haroon Basheer, *Student Member, IEEE*, Siddhant Dash, Jun Wen Wong, Jia Xu, Hoon Wei Lim, and Biplab Sikdar

***Abstract*—The critical and sensitive nature of data that the Internet-of-Things (IoT) devices produce makes them an attractive target for cyber attacks. Among the various types of attacks, malware is becoming a major concern for the IoT device. This article proposes a remote attestation protocol, hybrid remote attestation, which ensures the high availability of IoT devices during the software attestation process. The proposed attestation technique uses a randomized approach to attest different parts of an IoT device's memory. We use physical unclonable functions (PUFs) to protect the secrets of an IoT device from physical attacks. The security analysis shows that the proposed attestation technique can effectively detect roving malware. Implementation of the proposed protocol on Raspberry Pi and AVR/ARM-based ATMEL microcontrollers and comparison with existing techniques shows that the proposed protocol results in significantly higher availability and lower energy consumption.**

***Index Terms*—Internet of Things (IoT), malware, network security, remote attestation.**

I. Introduction

INTERNET of Things (IoT) and cyber–physical systems (CPSs) are increasingly being used in smart healthcare [1], industrial process control, smart cities, defense, etc. Therefore, it is crucial to protect the data and code of these embedded devices against malware. Attestation is the process of verifying the integrity of an embedded device's internal state to detect any unintended and malicious modification of the software running on an embedded device. Roving malware is considered the most difficult to detect. In this type of malware, the adversary avoids detection by either relocating itself in the memory or by deleting itself (and returning later). This article solves the issue of detecting roving malware.

Manuscript received October 17, 2019; revised January 21, 2020 and March 4, 2020; accepted March 17, 2020. Date of publication March 26, 2020; date of current version August 12, 2020. This work was supported in part by the National Research Foundation, Prime Minister's Office, Singapore, under its Corporate Laboratory@University Scheme, National University of Singapore and in part by the Singapore Telecommunications Ltd. *(Corresponding author: Muhammad Naveed Aman.)*

Muhammad Naveed Aman, Mohamed Haroon Basheer, and Siddhant Dash are with the Department of Computer Science, National University of Singapore, Singapore 117417 (e-mail: naveed@comp.nus.edu.sg; haroon.basheer@nus.edu.sg; siddhantdash@icloud.com).

Jun Wen Wong, Jia Xu, and Hoon Wei Lim are with the NUS-Singtel Cyber Security Research and Development Laboratory, National University of Singapore, Singapore 117417 (e-mail: junwen.wong@trustwave.com; jia.xu@trustwave.com; hoonwei.lim@trustwave.com).

Biplab Sikdar is with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore 117576 (e-mail: bsikdar@nus.edu.sg).

Digital Object Identifier 10.1109/JIOT.2020.2983655

Attestation techniques are generally divided into three categories: 1) software-based; 2) hardware-based; and 3) hybrid. Software-based attestation techniques exploit the computational limits of embedded devices, i.e., a certain algorithm can run within a certain time period. These techniques do not require any specialized hardware, making them suitable for low-cost embedded devices. However, these techniques are computationally complex and vulnerable to physical attacks. The attestation techniques which rely on specialized secure hardware such as trusted platform modules (TPMs) [2] or secure co-processors [3] are termed hardware-based. These techniques have low computational complexity and may also provide security against physical attacks. However, most IoT devices are simple and low cost in nature and cannot support these types of hardware modules. On the other hand, hybrid attestation techniques try to achieve the best of both worlds, i.e., a solution with lower computational complexity as compared to software-based attestation and the use of minimal hardware suitable for the low-end IoT devices. This article uses physical unclonable functions (PUFs) embedded into IoT devices as the sole extra hardware components. PUFs are an attractive choice for IoT devices because they are extremely cheap to manufacture and can support ultra high throughput with extremely low energy and silicon area footprints [4].

Availability is the degree to which an IoT device is unaffected by running a security service, i.e., the normal operation of the device is not affected. Given the safety-critical real-time applications that IoT devices are deployed in, it is crucial to design the security protocols which achieve high availability. Most of the existing attestation techniques require the attestation procedure to run without interruptions. As attestation procedures usually require a long time to complete, these techniques result in poor availability of IoT devices. To solve this problem, this article proposes a passive and lightweight attestation protocol for IoT devices named hybrid remote attestation (HAtt).

The trusted entity which initiates the attestation is called the verifier while the embedded device that proves the authenticity of its internal state is called the prover. Attestation techniques commonly work as follows: the verifier sends a challenge to the prover. The prover calculates a hash digest of its memory contents using the challenge and returns a response called the checksum to the verifier. The verifier uses the checksum to determine whether the prover is compromised or not. However, calculating the hash digest of the entire memory is a computationally intensive operation. The proposed attestation

2327-4662 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

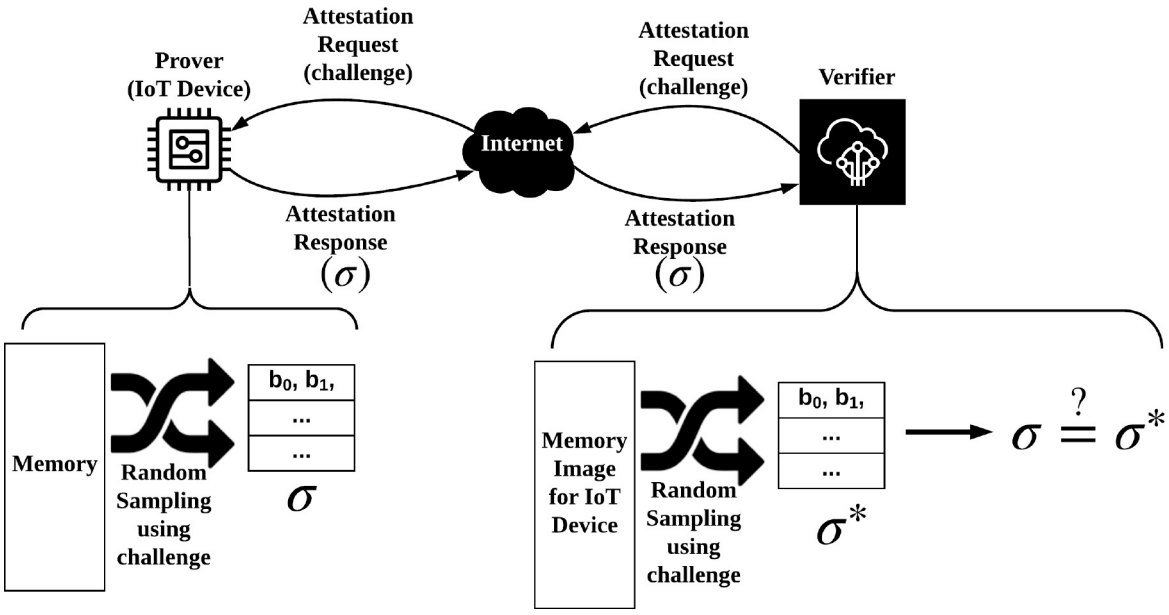


Fig. 1. Attestation process.

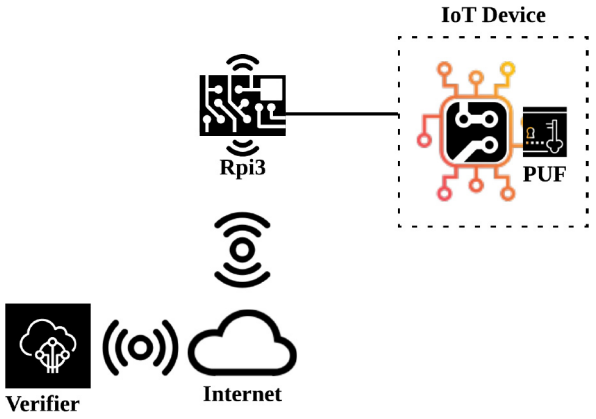


Fig. 2. System setup.

technique eliminates the need for calculating a complex checksum by using a randomized bit sampling approach as shown in Fig. 1. The system setup is shown in Fig. 2. We have an IoT device equipped with a PUF connected to a Raspberry PI (Rpi) module which in turn is connected to the Internet. Note that we are using the Rpi as a gateway (border router) to connect the IoT devices to the Internet. The verifier sends a challenge to the prover and the prover uses the challenge to carry out random sampling of its memory. These random bits are then sent to the verifier. The verifier uses a stored state of the prover's memory to perform the random sampling of bits. The verification bits sent by the prover (σ) are compared with the verification bits generated by the verifier (σ^*). The attestation is accepted if $\sigma = \sigma^*$.

The main contributions of this article are as follows.

1) An attestation technique which not only protects the IoT devices against the roving malware but also achieves high availability.
2) A lightweight privacy-preserving remote attestation protocol.
3) Experimental validation of the proposed protocol.

The remainder of this article is organized as follows. We discuss the related work in Section II, Section III provides the background and Section IV discusses our network model, assumptions, security requirements, and threat model. Section V presents the proposed attestation technique, and Section VI presents the proposed attestation protocol. Security analysis and simulation results are discussed in Section VII, and an informal security discussion is presented in Section VIII. The experimental validation is presented in Section IX and the performance of the proposed protocols is discussed in Section X. The conclusions of this article are presented in Section XI.

II. Literature Review

The prominent software-based attestation techniques in the existing literature include SWATT [8], SCUBA [9], and SAKE [10]. SWATT makes multiple iterations through the memory to calculate a collective hash. At least 50 000 iterations are required by SWATT to detect an adversary. This results in long execution times as well as complexity. Similarly, Seshadri *et al.* [9] used an approach like SWATT to detect infected nodes in a sensor network. In another similar work, Seshadri *et al.* [10] proposed an additional technique to establish a secure key between the sensor nodes after the detection of a compromised node. However, these techniques rely on accurate time measurements and complex operations. A recent protocol [11] for attestation calculates the checksum over partial memory to reduce the computational complexity. However, the attestation routine still requires to be run without any interruptions, resulting in low availability. Moreover, this technique requires a high number of iterations to detect malware.

Some of the popular hardware-based attestation techniques include [19]–[23]. The technique in [20] assumes each sensor

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF EXISTING ATTESTATION TECHNIQUES WITH HATT

Technique	[8]–[10], [25]–[27], [28]–[30], [31]–[33], [34]	[11]	[19]	[20], [21]	[5]	[6], [7]	[12], [35], [36]	[24]	HAtt
Computationally Complex Checksums	✓	✗	✗	✓	✓	✓	✓	✓	✗
Accurate Time Measurements	✓	✓	✗	✗	✓	✓	✓	✗	✗
Disabled Interrupts	✓	✓	✗	✗	✓	✓	✓	✗	✗
Require Advanced Hardware	✗	✗	✓	✓	✗	✓	✗	✓	✗
Privacy Concerns	✓	✓	✓	✓	✓	✓	✓	✓	✗
Require Homogeneous Devices	✗	✗	✓	✓	✗	✗	✓	✓	✗
Availability	✓	✓	✗	✗	✓	✓	✓	✗	✗

Computationally Complex Checksums: Does the technique calculate a hash digest of the entire memory?

Accurate Time Measurements: Does the technique rely on accurate execution times and network latency?

Disabled Interrupts: Does the technique disable all interrupts during its execution?

Require Advanced Hardware: Does the technique rely on costly hardware modules such as TPMs?

Privacy Concerns: Does the technique ensure IoT device anonymity?

Require Homogeneous Devices: Does the technique require all IoT devices to share the same architecture?

Availability: Is the normal operation of the device affected if the technique is executed?

node to be equipped with its own TPM. However, this may not be a realistic assumption given that many IoT devices are highly resource constrained. The techniques in [19] and [21] organize the nodes in a wireless sensor network (WSN) as clusters. One of the nodes in a cluster equipped with TPM is chosen as the cluster head. The cluster head verifies the integrity of the other nodes in its cluster. Similarly, the integrity of the cluster head may be verified using its TPM as the root of the trust. However, these techniques are vulnerable to a single point of failure, i.e., if a cluster head gets infected, the malware can easily propagate to other nodes. These techniques also require homogeneous devices equipped with TPMs not suitable for IoT devices.

The existing works on hybrid attestation include SMART [5], TrustLite [6], TyTan [7], and ATT-Auth [12]. However, these techniques fail to achieve the high availability of the IoT devices. Moreover, these techniques do not consider the roving malware in their threat model making them vulnerable to roving malware. A recently proposed hybrid attestation technique to detect roving malware is the SMARM [24]. SMARM uses the SMART architecture with one exception, i.e., it relaxes the atomicity requirement of SMART such that the attestation process can be interrupted. However, like software-based attestation techniques, SMARM computes a hash over the entire memory of an IoT device, thereby increasing the complexity and execution time. Moreover, due to the special requirements of the SMART architecture, SMARM cannot be applied to low-cost sensors. Thus, the existing literature on embedded device attestation suffers from one or more of the following issues.

1) *High overhead* for devices due to the use of complex operations.
2) Require *costly* secure hardware modules.
3) Require the interrupts to be disabled, thereby affecting the *availability* of the devices.
4) *Privacy concerns*.
5) Require *advanced hardware specifications*.

HAtt solves these issues using the following.

1) No complex operations are used on the IoT devices resulting in a lightweight mechanism.
2) Use of cheap PUFs to reduce the cost.
3) Higher availability by attesting the memory in parts instead of one go.
4) Pseudonym identities are used to provide the privacy of IoT devices.
5) High probability of detecting against roving malware.
6) Firmware of simple sensors can be verified without the need for special architectures.

To provide a comprehensive comparison of HAtt with the existing literature, Table I provides a summary.

III. INTRODUCTION TO PHYSICAL UNCLONABLE FUNCTIONS

Random changes in the manufacturing of embedded circuits lead to a complex physical structure allowing for a unique challenge–response mechanism. A PUF is characterized by a challenge–response-pair (CRP), i.e., $R = P(C)$, where R is the response to a challenge C by a PUF P. When a PUF is excited using the same challenge, it outputs a unique response. However, if a distinct PUF is presented

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

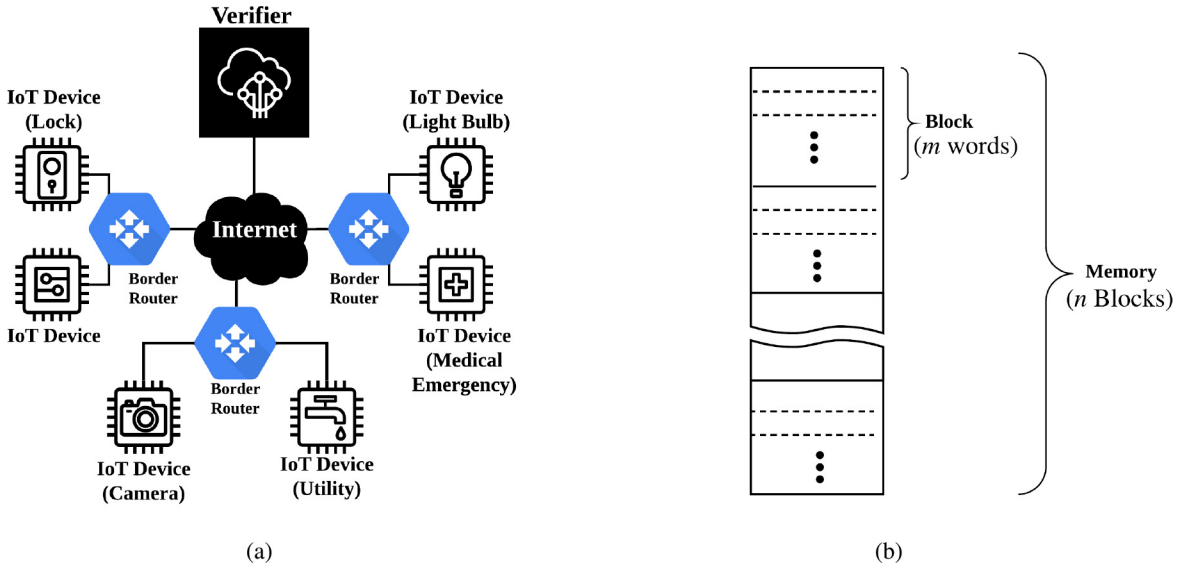


Fig. 3. System model. (a) Network model. (b) Memory divided into blocks.

with the same challenge, it will output a significantly different response. Thus, every PUF is unique in terms of its CRPs.

The response of a PUF can be affected by environmental variables, such as temperature and voltage. However, this issue can be prevented and stable PUF outputs suitable for security protocols can be achieved using fuzzy extractors [13], [14]. Therefore, we assume the use of ideal PUFs in this article. The PUFs come in different flavors, such as memory-based PUFs (using the random process variability in memory cells), delay-based PUFs (taking advantage of the variability between circuit delays), etc. PUFs form an effective way of protecting the IoT devices against the physical attacks by eliminating the need to store secret keys in an IoT device's memory.

IV. Network Model, Assumptions, and Threat Model

A. Network Model

The network model considered in this article is shown in Fig. 3(a). IoT devices are connected to the Internet through border router elements (e.g., 6LoWPAN routers). The IoT devices communicate with the verifier in the data center over the Internet.

B. Assumptions

We make the following assumptions.

1) Every IoT device has a PUF and is considered a System on Chip (SoC). The PUF is assumed to be useless and destroyed if separated from the IoT device [15], [16].
2) The microcontroller and the PUF form an SoC and the communication between them is considered secure [15], [16].
3) The IoT devices have limited memory and processing power.
4) The verifier is considered trusted and is not constrained in terms of resources.
5) The attestation code is immutable.
6) Execution of the attestation code when measuring a single memory block cannot be interrupted.
7) Table II gives the set of notations used in this article.

TABLE II
Notations

Notation	Description
$\boldsymbol{\sigma}$	Attestation response / Verification bits
b_i	i'th bit of a memory word
Rpi3	Raspberry Pi 3
PUF	Physical Unclonable Function
m	Number of words per block
n	Total number of blocks in memory
β	Number of verification bits read per memory word
V	Verifier
P	Prover
t_i	i'th time interval
$\boldsymbol{S}$	Memory State
S_β, S_ω	Random seeds
ρ	random sequence for attesting blocks
ω	Number of bits per word in memory
Ω	Sequence of random numbers uniformly distributed between 1 and ω
M_{ρ_i}	Memory block to be attested at time t_i
P_D	Probability of detecting an adversary
P_{Adv}	Probability of an adversary dodging detection
M_{Δ_i}	block containing the adversary at time t_i
$p_{\zeta_i}^{move}$	Probability of an adversary getting caught after moving to another block after time t_i
$p_{\zeta_i}^{move}$	Probability of getting caught if the adversary decides to stay at its current block after time t_i
ID_i	ID of the IoT device
$H(X)$	Hash of X
$\|$	Concatenation operator
$\{M\}_k$	Message M is encrypted using key k
PID_A^i	Pseudonym identity of IoT device ID_A for the i-th iteration
C^i	Challenge for the i-th iteration
R^i	Response of the respective PUF for C^i

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

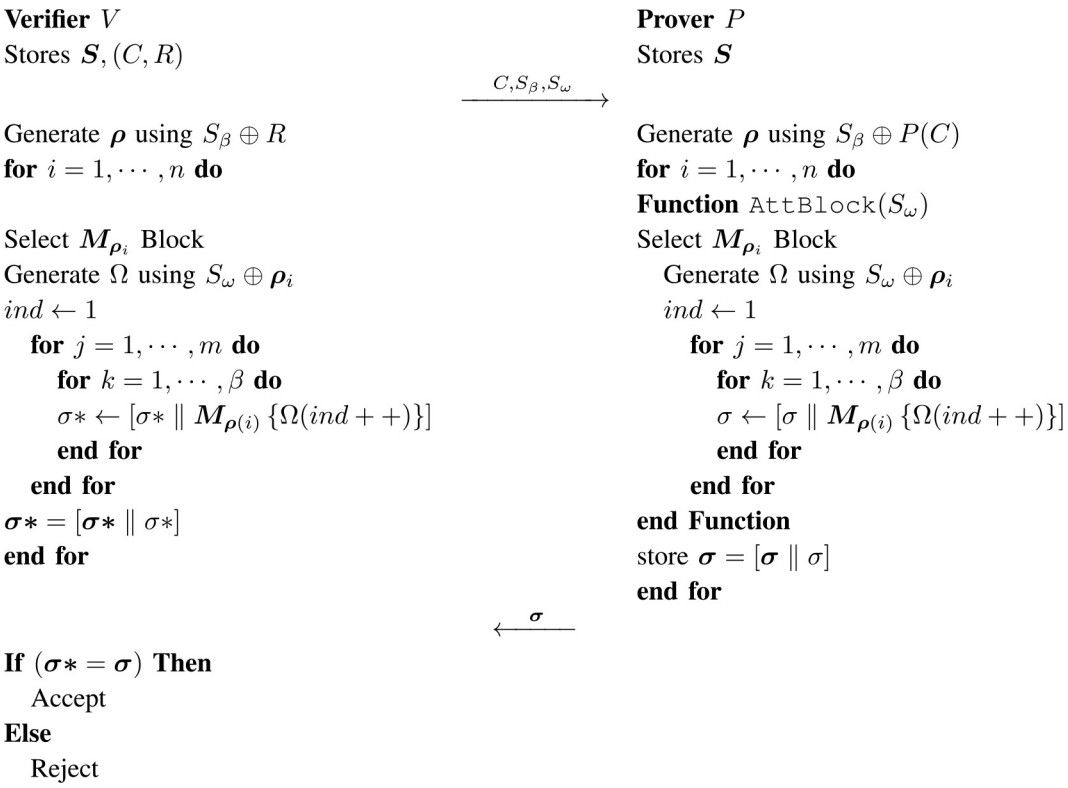


Fig. 4. Proposed software attestation technique.

C. Threat Model

The adversary (*Adv*) intends to stay in the IoT device's memory and avoid detection. HAtt divides the memory of an IoT device into blocks as shown in Fig. 3(b). The *Adv* occupies at least one memory block. The *Adv* can interrupt and make changes to any memory block anytime between runs of the attestation code on a single memory block. We assume that the *Adv* remains on the device all the time and cannot delete itself. We also assume that the adversary has knowledge of: 1) the time when attestation was initiated and 2) the time to measure one memory block. Note that the adversary can use this knowledge to calculate the size of memory not yet attested.

The adversary may inject, replay, tamper, and eavesdrop on packets sent by an IoT device. We also assume that an adversary may gain physical access to an IoT device and subject it to physical attacks to extract stored secrets. This set of attacks can be modeled with the following set of queries.

1) `Send`*S*(*S*, `m0`, `r0`, `m1`) is used to model the query where the attacker $\mathcal{A}$ acts like a legitimate IoT device and sends a message `m0` to *S* and receives `r0`. The IoT device then replies to *S* with `m1`.
2) `Send`ID(ID, `m0`, `r0`) is used to model the query where the attacker $\mathcal{A}$ acts like a server and sends a message `m0` to an IoT device ID and receives `r0`.
3) `Monitor`(ID,*S*) is used to model the attacker's ability to continuously eavesdrop and monitor the radio channel between the IoT device ID and *S*.
4) `Drop`($\mathcal{A}$) is used to model the attacker's ability to drop packets between ID and *S*. This query may be used by the attacker to launch a DoS attack, i.e., dropping selective packets to interrupt the synchronization between the two entities.
5) `Reveal`(ID) is used to model the attacker's ability to launch a physical attack on an IoT device and obtain the secrets stored in its memory.

The attacker $\mathcal{A}$ can call `Send`*S*, `Send`ID, `Monitor`, and `Drop` any polynomial number of times. However, as mentioned in the assumptions, any physical tampering with an IoT device renders it useless. Therefore, $\mathcal{A}$ may call `Reveal` only once.

V. Proposed Attestation Technique

We denote the prover or the IoT device proving the integrity of its software by *P*. The verifier or the trusted server verifying the validity of the IoT device's software is denoted by *V*. The memory to be attested is divided into *n* blocks with *m* words per block as shown in Fig. 3(b).

The attestation process as shown in Fig. 4 continues as follows.

1) *V* sends a random challenge *C* along with two random seeds S_β and S_ω to the prover.
2) *P* generates the response *R* for *C* using its PUF and uses $S_\beta \oplus R$ as a seed to generate the next random permutation of block numbers to be attested, $\boldsymbol{\rho} = \boldsymbol{\rho}_1, \boldsymbol{\rho}_2, \ldots, \boldsymbol{\rho}_n$.
3) The routine `AttBlock`(S_ω) is called to attest each block. Execution of this routine (when measuring a

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

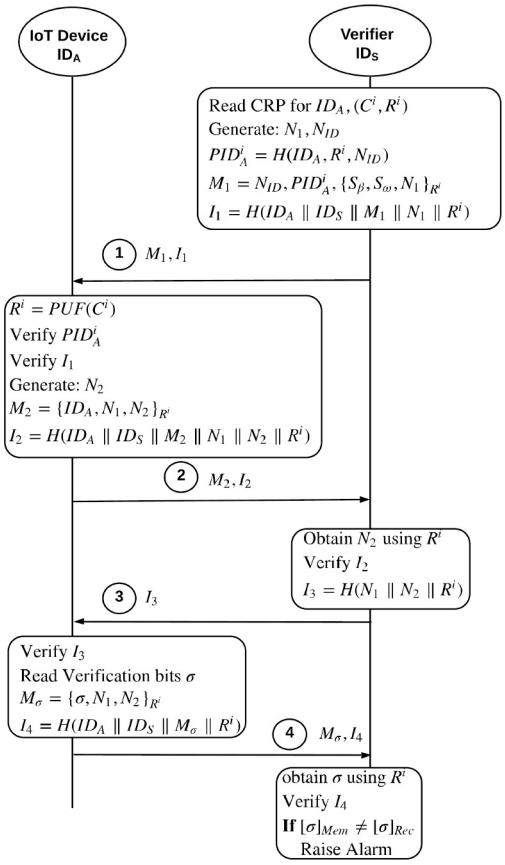


Fig. 5. Proposed protocol for IoT device attestation.

single memory block) cannot be interrupted. When a block $\boldsymbol{M}_{\boldsymbol{\rho}_i}$ is selected for attestation, P uses $S_\omega \oplus \boldsymbol{\rho}_i$ as the seed for selecting β random bits within each word of the block. For example, for each of the m words in a block, the attestation code selects β bits randomly. P generates a sequence of random numbers Ω uniformly distributed between 1 and ω, where ω is the size of a word in bits, i.e., $\Omega = U[1, \omega]^{m\beta}$ for each block. Using Ω, $m\beta$ bits are read from $\boldsymbol{M}_{\boldsymbol{\rho}_i}$ and stored in the memory by appending them to a bitstring $\boldsymbol{\sigma}$.

4) Steps 3) and 4) are repeated until all the blocks are attested.
5) P sends $\boldsymbol{\sigma}$ to V.
6) V uses its stored copy of the state of the provers memory $\boldsymbol{M}$ to verify $\boldsymbol{\sigma}$.

The verifier may run multiple iterations of the above attestation procedure to improve the detection probability.

VI. Proposed Attestation Protocol

The server stores an initial CRP (C^i, R^i) for each IoT device. The initial parameters are obtained by the server using a time-based one-time password algorithm (TOTP) [18] and an operator using a password. Each IoT device stores the initial challenge C^i.

The steps of HAtt as shown in Fig. 5 are as follows.

1) The verifier reads the CRP for the IoT device ID_A and generates random numbers N_1 and N_{ID}. The verifier then uses N_{ID} to generate a pseudonym identity for the IoT device ID_A, i.e., $\mathrm{PID}_A^i = H(\mathrm{ID}_A, R^i, N_{\mathrm{ID}})$. The verifier then sends the message $M_1 = N_{\mathrm{ID}}, \mathrm{PID}_A^i, \{S_\beta, S_\omega, N_1\}_{R^i}$ along with the authentication parameter I_1 to the IoT device ID_A. Note that the verifier uses the pseudonym identity PID_A^i of the IoT device for anonymity. Moreover, the authentication parameter is used to verify the integrity of the message being sent. If the receiver cannot validate the authentication parameter then the protocol is terminated. The authentication parameters are always constructed using a fresh nonce (in this case N_1) to avoid replay attacks. We use a similar approach throughout this article.
2) The IoT device ID_A obtains the PUF response R^i using its PUF and the challenge C^i. It then verifies I_1 and the pseudonym identity PID_A^i making sure the packet is addressed to itself and not tampered. It then generates a random number N_2 and sends the Message $M_2 = \{\mathrm{ID}_A, N_1, N_2\}_{R^i}$ along with the corresponding authentication parameter I_2 to the verifier.
3) The verifier decrypts M_2 to obtain N_2 and verifies I_2. If verification fails, the protocol is terminated. Otherwise, the verifier accepts the authentication of the IoT device ID_A and sends an authentication parameter I_3 as an acknowledgment.
4) The IoT device ID_A verifies I_3. It then reads the verification bits $\boldsymbol{\sigma}$ and sends them to the server in the encrypted message $M_{\boldsymbol{\sigma}} = \{\sigma, N_2, N_2\}_{R^i}$ along with the corresponding authentication parameter.
5) The server decrypts $M_{\boldsymbol{\sigma}}$ to obtain $\boldsymbol{\sigma}$. It then verifies I_4 and accepts attestation if $\boldsymbol{\sigma}*$ read from the verifier's memory is equal to $\boldsymbol{\sigma}$ received from the IoT device ID_A as described in Section V.

VII. Security Analysis

In this section, we present a formal security analysis of the proposed attestation technique described in Section V and the security verification simulations for the proposed attestation protocol described in Section VI.

A. Security Proofs for the Proposed Attestation Technique

The *Adv* is assumed to occupy a single block of memory. Moreover, we assume that it is active during attestation, i.e., it can interrupt the attestation procedure after each block and change the contents of any block of M. However, it cannot change anything when the attestation procedure is measuring a block. The *Adv*'s objective is to remain in M without being detected. Thus, to be successful, the *Adv* has to relocate itself and restore the contents of the block where it resides before it is measured.

As discussed in Section IV-C, the only piece of information that *Adv* might have is the size of M that has not yet been measured. It is clear that if *Adv* remains in the same block all the time then it cannot avoid detection. Therefore, it must relocate itself at least once.

Theorem 1: The optimal strategy for *Adv* is to relocate itself after every memory block is measured, i.e., $n - 1$ times. The

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

probability of detection in this case is given by

$$P_D = 1 - e^{-1}. \tag{1}$$

Proof: If we have n blocks in memory, then to complete one iteration of the attestation procedure, we need n time intervals, i.e., $t_1, t_2, \ldots, t_n$. In each time interval, one block is attested identified by the random sequence $\boldsymbol{\rho}_1, \boldsymbol{\rho}_2, \ldots, \boldsymbol{\rho}_n$. Let M_{Δ_i} represent the block containing *Adv* at the time t_i, for $1 \leq M_{\Delta_i} \leq n$ and $1 \leq i \leq n$.

If *Adv* evades detection in t_i, then it can either stay at its current block or move to another block during t_{i+1}. Assume M_x^* and $\boldsymbol{\zeta}$ represent a memory block x that has been previously measured and the event of *Adv* getting caught, respectively. The probability of getting caught, if the adversary moves to another block after t_i is given by

$$\begin{aligned} p_{\zeta_i}^{\text{move}} &= \Pr\left[M^*_{\Delta_{i+1}}\right]\Pr\left[\boldsymbol{\zeta} \mid M^*_{\Delta_{i+1}}\right] \\ &\quad + \Pr\left[M_{\Delta_{i+1}}\right]\Pr\left[\boldsymbol{\zeta} \mid M_{\Delta_{i+1}}\right] \qquad (2) \\ &= \frac{i}{n} \cdot 0 + \frac{n-i}{n}\frac{1}{n-i}\left(1 - \left(\frac{1}{2}\right)^{\gamma}\right) \\ &= \frac{1}{n}\left(1 - \left(\frac{1}{2}\right)^{\gamma}\right) \qquad (3) \end{aligned}$$

where $\gamma = m\beta$, and m and β are the number of words per block and the number of transmitted bits per word, respectively. Let

$$\Upsilon = 1 - \left(\frac{1}{2}\right)^{\gamma}. \tag{4}$$

Then, (3) can be written as

$$p_{\zeta_i}^{\text{move}} = \frac{\Upsilon}{n}. \tag{5}$$

Similarly, the probability of getting caught, if the *Adv* decides to stay at its current block after t_i can be given by

$$\begin{aligned} p_{\zeta_i}^{\text{stay}} &= \Pr\left[M^*_{\Delta_j}\right]\Pr\left[\boldsymbol{\zeta} \mid M^*_{\Delta_j}\right] + \Pr\left[M_{\Delta_j}\right]\Pr\left[\boldsymbol{\zeta} \mid M_{\Delta_j}\right] \qquad (6) \\ &= \frac{j}{n} \cdot 0 + \left(1 - \frac{j}{n}\right)\left(\frac{1}{n-i}\right) \cdot \Upsilon \\ &= \frac{\Upsilon(n-j)}{n(n-i)} \qquad (7) \end{aligned}$$

where j denotes the last interval when *Adv* moved. For example, if *Adv* moved after t_2, then $j = 2$. From (5) and (7), if $j < i$, then $p_i^{\text{stay}} > p_i^{\text{move}}$. Thus, the optimal strategy for *Adv* to evade detection is to move after every interval. The probability of dodging detection can be given by

$$P_{\text{Adv}} = \left(1 - p_{\zeta_i}^{\text{move}}\right)^{n-1} \cdot \Pr\left[M_{\Delta_1} \neq \rho_1\right]. \tag{8}$$

We observe that $\Upsilon \approx 1$ for $\gamma = m\beta \geq 8$, therefore

$$P_{\text{Adv}} = \left(1 - \frac{\Upsilon}{n}\right)^{n-1}\left(1 - \frac{1}{n}\right) \approx e^{-1} \tag{9}$$

where the probability that M_{Δ_1} is not the first block measured is represented by $\Pr[M_{\Delta_1} \neq \boldsymbol{\rho}_1]$. Note that if $M_{\Delta_1} = \boldsymbol{\rho}_1$, then the *Adv* has no chance of evading detection. The probability of detection is given by

$$P_D = 1 - P_{\text{Adv}} = 1 - e^{-1}. \tag{10}$$

■

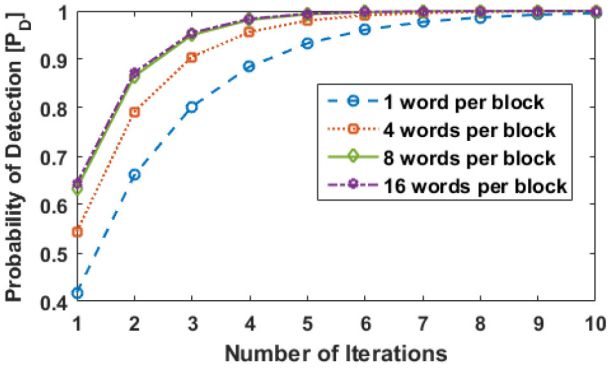


Fig. 6. Probability of detection for $n = 16$, $\beta = 1$, and $m = 1, 4, 8, 16$.

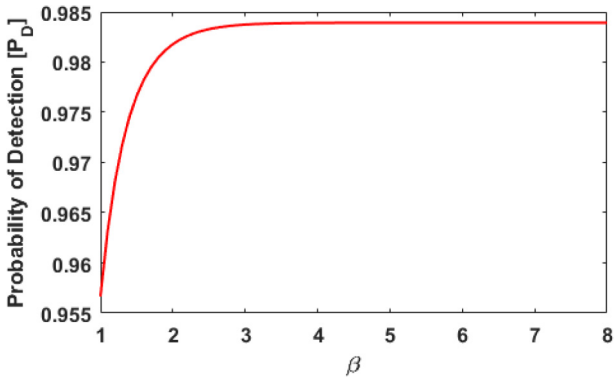


Fig. 7. Probability of detection for $n = 16$, $m = 4$, and four iterations.

To improve the probability of detection, we can make multiple iterations through the memory being attested. Fig. 6 shows that P_D goes beyond 95% using only three iterations and four words per block. P_D can also be improved by increasing β, i.e., the number of bits read for each word in a block, as shown in Fig. 7.

If the *Adv* needs to occupy more than one block, i.e., it is larger than one block, then we can consider the probability of detection as the probability of detecting multiple separate single-block adversaries, i.e.,

$$P_D(J = j) = P_D^j = \left[1 - e^{-1}\right]^j \tag{11}$$

where j is the number of blocks occupied by *Adv*. Fig. 8 shows that even with a few words per block and just one iteration, if the adversary occupies multiple blocks, P_D improves with the increase in blocks infected by the *Adv*.

B. Security Proofs for the Proposed Attestation Protocol

This section presents a formal security analysis of the proposed protocol shown in Fig. 5.

Lemma 1: The response of a PUF cannot be predicted.

Proof: A PUF can be represented as $\{0,1\}^{l_1} \rightarrow \{0,1\}^{l_2}$, i.e., given a challenge of the length l_1, it outputs a response of the length l_2. Let us model the security of a PUF with

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

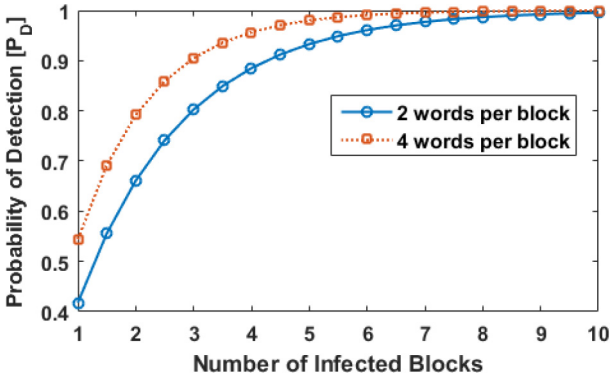


Fig. 8. Probability of detection for $n = 16$, $\beta = 1$, and one iteration.

a security game $\text{Exp}^{\text{Sec}}_{PUF,\mathcal{A}}$ between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ as follows.

1) $\mathcal{A}$ sends a randomly chosen challenge C^i to $\mathcal{C}$. $\mathcal{C}$ uses the PUF to reveal R^i to $\mathcal{A}$.
2) $\mathcal{C}$ chooses another random challenge C^x (not used before) and obtains the response R^x with the PUF, i.e., $R^x = PUC(C^x)$.
3) $\mathcal{A}$ is allowed to query the PUF a polynomial number of times for challenges other than C^x.
4) $\mathcal{A}$ outputs its guess $R^{x'}$ for the challenge C^x and wins the game if $R^{x'} = R^x$.

The advantage of the adversary $\mathcal{A}$ in this game is given by $\boldsymbol{\alpha}^{\text{PUF}}_{\mathcal{A}} = \texttt{Pr}[R^{x'} = R^x]$. Every PUF produces a unique response and cannot be cloned [17]. Thus, the adversary can only guess the output of a PUF to a given challenge. Therefore, $\boldsymbol{\alpha}^{\text{PUF}}_{\mathcal{A}} = (1/2^{l_2})$. ■

Lemma 2: The secrets used in the proposed protocol cannot be revealed even after calling the `Reveal` oracle.

Proof: The IoT device does not store any secrets in its memory. Therefore, the adversary cannot obtain the secret response R^i even after calling the `Reveal` oracle. Note that $\mathcal{A}$ cannot use C^i to obtain R^i because of the SoC assumption. ■

Lemma 3: The pseudonym identities of an IoT device cannot be correlated in the proposed protocol even after calling the `Reveal` oracle.

Proof: The pseudonym identity PID^i_{ID} of an IoT device is constructed as $H(\text{ID}_A, R^i, N_{\text{ID}})$, i.e., using a random nonce N_{ID} which is refreshed after each round. Therefore, each pseudonym identity is valid for only a single round. This shows that the adversary cannot correlate the pseudonym identity for the current round with that of the next or previous round unless he/she can obtain the secret response R^i. However, according to Lemma 2, this is not possible. Thus, the adversary can only guess the secret response R^i and the advantage of the adversary in this case can be modeled as $\boldsymbol{\alpha}^{\text{ID}}_{\mathcal{A}} = \texttt{Pr}[\texttt{Corr}(\text{PID}^i, \text{PID}^{i+1}) \neq 0] = \boldsymbol{\alpha}^{\text{PUF}}_{\mathcal{A}} = (1/2^{l_2})$, where `Corr` represents the correlation coefficient. ■

Theorem 2 (Mutual Authentication): If an IoT device successfully completes a run of the protocol, it has indeed done so with the legitimate verifier. Similarly, if the verifier has successfully completed a run of the protocol with an IoT device, it has indeed done so with the legitimate device.

Proof: The adversary may try to impersonate as a legitimate IoT device and authenticate itself to the verifier. The following game between $\mathcal{C}$ and $\mathcal{A}$ is used to model this attack.

1) $\mathcal{C}$ selects a legitimate IoT device ID_1 and initiates the proposed protocol between the IoT device ID_1 and the verifier.
2) $\mathcal{A}$ calls `SendS`, `SendID`, `Monitor`, and `Drop` a polynomial number of times on the verifier and the IoT device ID_1.
3) $\mathcal{A}$ invokes the `SendS` oracle to authenticate itself as a legitimate IoT device to the verifier.
4) $\mathcal{A}$ wins the game if he/she can successfully complete the proposed protocol.

$\mathcal{A}$ can only successfully authenticate itself if it can produce the correct authentication parameter I_2 in Message 2 of the proposed protocol. To do so, $\mathcal{A}$ needs R^i. Assume $\mathcal{A}$ can reveal l'_2 bits (out of l_2 bits) in R^i. Then, the advantage of $\mathcal{A}$ in revealing R^i is given by $\texttt{Pr}[R^{i'} = R^i] = (1/2^{l_2 - l'_2})$. We can model the adversary's advantage for successfully authenticating itself to the server as $\boldsymbol{\alpha}^{\text{Auth1}}_{\mathcal{A}} = \texttt{Pr}[R^{i'} = R^i] - \boldsymbol{\alpha}^{\text{PUF}}_{\mathcal{A}}$. However, by Lemmas 1 and 2, $\mathcal{A}$ can only randomly guess R^i, i.e., $l'_2 = 0$ and $\texttt{Pr}[R^{i'} = R^i] = (1/2^{l_2}) = \boldsymbol{\alpha}^{\text{PUF}}_{\mathcal{A}}$. Thus, $\boldsymbol{\alpha}^{\text{Auth1}}_{\mathcal{A}} = 0$.

$\mathcal{A}$ may also try to authenticate itself to an IoT device as a verifier. If we denote the advantage of the adversary in this attack by $\boldsymbol{\alpha}^{\text{Auth2}}_{\mathcal{A}}$. Using a similar approach as above, it is trivial to show that $\boldsymbol{\alpha}^{\text{Auth2}}_{\mathcal{A}} = 0$. ■

Theorem 3 (Privacy): In the proposed protocols, the IoT devices are untraceable.

Proof: If $\mathcal{A}$ cannot correlate two successful runs of the the proposed protocol by the same IoT device with the verifier, then the proposed protocol is termed untraceable. This can be modeled by the following security game.

1) $\mathcal{C}$ selects two IoT devices ID_1 and ID_2 and uses each IoT device to run the proposed protocol with the verifier.
2) $\mathcal{A}$ calls `SendS`, `SendID`, `Monitor`, and `Drop` a polynomial number of times on the verifier and the IoT devices.
3) $\mathcal{C}$ randomly picks one of the IoT devices ID$*$.
4) $\mathcal{A}$ calls `SendS`, `SendID`, `Monitor`, and `Drop` a polynomial number of times on the verifier and IoT device ID$*$.
5) $\mathcal{A}$ reveals her/his guess ID$'$.
6) $\mathcal{A}$ wins the game if $\text{ID}' = \text{ID}*$.

The adversary's advantage of a successful guess for ID$'$ can be modeled as $\boldsymbol{\alpha}^{Pri1}_{\mathcal{A}} = 2 \times (\texttt{Pr}[\text{ID}' = \text{ID} *] - [1/2])$. By Lemma 3, the advantage of $\mathcal{A}$ in correlating the pseudonym identities of ID$*$ can be modeled as $\boldsymbol{\alpha}^{Pri2}_{\mathcal{A}} = \texttt{Pr}[\texttt{Corr}(\text{PID}^i, \text{PID}^{i+1}) \neq 0)]$. The advantage of the adversary in winning this game can be modeled as $\boldsymbol{\alpha}^{Pri}_{\mathcal{A}} = \boldsymbol{\alpha}^{Pri1}_{\mathcal{A}} + \boldsymbol{\alpha}^{Pri2}_{\mathcal{A}} - \boldsymbol{\alpha}^{Pri1}_{\mathcal{A}} \times \boldsymbol{\alpha}^{Pri2}_{\mathcal{A}}$. If an adversary makes a random guess for ID$*$ then he/she has no advantage, i.e., $\texttt{Pr}[\text{ID}' = \text{ID} *] = (1/2)$. Using Lemmas 1–3, we can conclude that $\boldsymbol{\alpha}^{Pri}_{\mathcal{A}} = 0$. ■

Lemma 4: The proposed protocols are secure against DoS attacks.

Proof: An adversary $\mathcal{A}$ may attempt at breaking the synchronization between an IoT device and the verifier by

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

dropping/blocking packets carrying important parameters, e.g., pseudonym identities. However, in the proposed protocol, the verifier generates the pseudonym identity dynamically and even if the adversary drops any message in the proposed protocol, the verifier can still carry out a new run of the protocol and authenticate itself to the IoT device without any issues. ■

Lemma 5: The proposed protocols are safe against the physical and cloning attacks.

Proof: The IoT devices do not store any secrets in their memory. Moreover, with the PUF and the microcontroller being an SoC, an adversary can neither separate the PUF from the microcontroller nor can it eavesdrop on the communication between the PUF and the microcontroller [15], [16]. Thus, according to Lemmas 1 and 2, we can conclude that the proposed protocol is safe against the physical and cloning attacks. ■

C. Security Verification of HAtt

The security properties of the proposed protocol were verified using the automated security verification tool ProVerif (PV) [39]. In general, a security protocol can be implemented in PV with three parts: 1) the cryptographic primitives are modeled using declarations; 2) the subprocesses are defined using process *macros*; and 3) the *main* process to instantiate subprocesses and encode the protocol. Arbitrarily many sessions were simulated between the IoT devices and the verifiers which were instantiated as separate subprocesses. The implementation source code and output can be found in the PV simulation scripts available in [38].

PV is used to check the proposed protocol against the following attacks [40]–[42].

1) *Impersonation and Replay Attacks:* To establish the authentication properties, the following events are defined in PV.
   a) *event beginAfull*($\mathrm{ID}_A, \mathrm{ID}_S, N_1, N_2$) used by the verifier to record the belief IoT device ID_A has initiated a run of the protocol with the verifier using N_1 and N_2 as the secret parameters.
   b) *event endAfull*($\mathrm{ID}_A, \mathrm{ID}_S, N_1, N_1$) which means that the IoT device ID_A has successfully completed the protocol with the verifier usin N_1 and N_2 as the secret parameters.
   c) *event beginBfull*($\mathrm{ID}_A, \mathrm{ID}_S, N_1, N_2$) represents IoT device ID_A's intention to launch the protocol with the verifier with the given protocol parameters.
   d) *event endBfull*($\mathrm{ID}_A, \mathrm{ID}_S, N_1, N_2$), which records the verifier's belief of successfully completing the protocol with the IoT device ID_A with the given protocol parameters.

   The following queries are then used in PV to check the proposed protocol's resilience against impersonation, replay, and data tampering attacks:

$$\texttt{query}\ inj-event(endBfull(\cdots)) ==> inj-event(beginBfull(\cdots)) \quad (12)$$

$$\texttt{query}\ inj-event(endAfull(\cdots)) ==> inj-event(beginAfull(\cdots)). \quad (13)$$

   where the statement E_A ==> E_B in PV is used to check the fact "if an event E_A has been executed, then it has been preceded by the execution of event E_B previously."
2) *Secrecy for IoT Device ID_A and the Server:* To check whether a secret can be revealed by an adversary, PV uses `query attacker(X)`, i.e., the reachability property of `X` is tested using this query. A true result shows that the adversary cannot reveal X. The secrecy of a term is established by using it as a session key to encrypt an arbitrary term M and then checking whether an attacker can successfully obtain M. Using such queries, we have shown that the adversary cannot reveal any of the secrets N_1, N_2, S_β, S_ω, σ, and ID_A.
3) *Strong Secrecy for IoT Device ID_A and the Server:* Observational equivalence is validated in PV using the `noninterf` queries. Such queries have been used to establish the strong secrecy of the exchanged secrets.
4) *Strong Anonymity:* The proposed protocol uses pseudonym identities to achieve anonymity of the IoT devices. To verify strong anonymity in PV we used the `noninterf` query, i.e., we encrypted an arbitrary term `ID_A` using the identity of IoT device A, i.e., ID_A as the session key. Then, the `noninterf` query is used to check whether the attacker can reveal the identity of the IoT device ID_A.
5) *Ephemeral Secret Leakage (ESL):* The proposed protocol is based on the CK-adversary model [44]. Under the CK-adversary model, the adversary is capable of revealing the session state, private, and session keys in addition to the capabilities under the DY model [43]. To check the proposed protocol for ESL in PV, we used the following queries to test the short-term secrets N_1 and N_2: `query attacker(new N_1); attacker(new N_2)`. Adding the `new` keyword to the attacker query in PV can be used to verify the ESL attacks.

The output after running our PV script is shown in Fig. 9. In this output, `ID_A` represents the identity of the IoT device ID_A. `ASW_t`, `ASB_t`, and `ASigma_t` are used to verify the secrecy of S_ω, S_β, and σ, respectively, on the site of the principal IoT device ID_A, i.e., these arbitrary terms are encrypted using the respective secrets and we check whether an attacker can obtain these terms from the site principle of the IoT device. Similarly, `BSW_t`, `BSB_t`, and `BSigma_t` are used to verify the secrecy of S_ω, S_β, and σ, respectively, on the site principal of the verifier. Note that all the security tests have been passed successfully.

VIII. Informal Security Analysis

In this section, we discuss the proposed protocol's resilience against various known attacks.

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

```
muhammad@ubuntu:~/proverif1.94pl1/research/Attestation 2$ proverif Att_IoT3.pv | grep "RES"
RESULT inj-event(endAfull(x1,x2,x3,x4)) ==> inj-event(beginAfull(x1,x2,x3,x4)) is true.
RESULT inj-event(endAparam(x,y)) ==> inj-event(beginAparam(x,y)) is true.
RESULT not attacker_nonce(N1[hostX = v_3901,!1 = v_3902]) is true.
RESULT not attacker_nonce(N2[MACX = v_3896,m_54 = v_3897,PID_X = v_3898,NID = v_3899,!1 = v_3900]) is true.
RESULT not attacker_bitstring(ID_A[]) is true.
RESULT Non-interference BSW_t is true (bad not derivable).
RESULT Non-interference BSB_t is true (bad not derivable).
RESULT Non-interference ASW_t is true (bad not derivable).
RESULT Non-interference ASB_t is true (bad not derivable).
RESULT Non-interference BSigma_t is true (bad not derivable).
RESULT Non-interference ASigma_t is true (bad not derivable).
RESULT Non-interference ID_A is true (bad not derivable).
```

Fig. 9. PV output.

A. Mutual Authentication Between the IoT Device and the Verifier

The verifier authenticates the IoT device ID_A using the verification parameter $I_2 = H(\mathrm{ID}_A, \mathrm{ID}_S, M_2, N_1, N_2, R^i)$. Only the legitimate IoT device can form a valid I_2 because only it can obtain R^i using its PUF. Similarly, the IoT device ID_A authenticates the verifier using the authentication parameter $I_1 = H(\mathrm{ID}_A, \mathrm{ID}_S, M_1, N_1, N_2, R^i)$, i.e., only the legitimate verifier knows R^i and can form a valid I_1.

B. Anonymity and Traceability

Anonymity is critical for the IoT systems, i.e., it is important to keep the identity of IoT devices secret. For this purpose, the proposed protocol employs pseudonym identities $\mathrm{PID} = H(\mathrm{ID}, R^i, N_{\mathrm{ID}})$. Given the PID of an IoT device, an attacker cannot reveal the true identity of the device as he/she cannot obtain the secret PUF response R^i. Moreover, a new random nonce N_{ID} is used in each new run of the protocol. Therefore, it is not possible for an adversary to correlate two different PIDs of the same IoT device. Thus, the IoT devices in the proposed protocol can be considered untraceable.

C. DoS Attacks

The synchronization of the IoT device and the server in the proposed protocol does not rely on the transfer of any critical secrets. Therefore, even if an adversary drops a message in the proposed protocol, the verifier can still initiate a new run of the protocol. Thus, the proposed protocol can be considered secure against DoS attacks.

D. Replay Attacks

An attacker may try to delay or repeat previously transmitted packets to compromise the security of a protocol. The proposed protocol uses a new random nonce in the authentication parameters in each message and run of the protocol to avoid such attacks.

E. Physical and Cloning Attacks

The use of PUFs in the proposed protocol ensures security against the physical and cloning attacks. An adversary cannot extract any secrets from an IoT device's memory neither can he/she clone an IoT device due to the unclonable property of PUFs.

TABLE III
LATENCY FOR HATT, NUMBER OF ITERATIONS = 4, AND PROBABILITY OF EVASION < 5%

m	β	Latency (Sec)
4	1	0.104
	2	0.106
	4	0.107
8	1	0.107
	2	0.107
	4	0.108
16	1	0.107
	2	0.110
	4	0.119

TABLE IV
LATENCY OF EXISTING ATTESTATION TECHNIQUES, PROBABILITY OF EVASION < 5%

Technique		Latency (sec)
SWATT [8]	50,000 Iterations	80.97
	150,000 Iterations	288.30
	250,000 Iterations	481.77
[11]		0.855
TyTan [7]		0.126
SMARM [24]		53.06

IX. EXPERIMENTAL VALIDATION

Considering availability as an important objective of the proposed attestation protocol, we need to evaluate the latency for running the proposed attestation routine on an IoT device. For this purpose, we used the Rpi 3 (Rpi3) platform with sensors connected to it via the serial port. The sensors are based on the Arduino Due development board (ATSAM3X8E) and the Arduino Mega board (ATMega2560). The data are gathered at a rate of one sample per 60 s. The memory used for attestation is 512 kB in size. The experimental setup is shown in Fig. 10. In Fig. 10, we have two Rpi3s, one is connected to the Arduino Due development board while the other is connected to the Arduino Mega board. Both the Rpi3s

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ENERGY CONSUMPTION FOR PROPOSED TECHNIQUE COMPARED TO EXISTING SOFTWARE-BASED ATTESTATION TECHNIQUES FOR A MEMORY SIZE OF 128 KB

Key Size	Proposed Technique (μJ)			SWATT (μJ)			Savings (μJ)	[11] (μJ)			Savings (μJ)
	Protocol	Other	Total	Protocol	Other	Total		Protocol	Other	Total	
CPU											
128 bits	743,136	94,448	837,585	2,880,275	264,602	3,144,878	2,307,293 (73%)	1,263,331	127,504	1,390,835	553,250 (40%)
192 bits	949,646	111,815	1,061,461	4,129,846	368,796	4,498,642	3,437,180 (76%)	1,661,880	156,541	1,818,421	756,960 (42%)
256 bits	1,156,158	129,181	1,285,340	5,379,327	471,258	5,850,585	4,565,244 (78%)	2,081,084	187,312	2,268,396	983,056 (43.5%)
Radio											
128-bits	124,239	1,044,019	1,168,258	395	3,971,376	3,971,772	2,803,513 (70%)	256,242	2,153,289	2,409,531	1,241,273 (51.5%)
192 bits	124,239	1,332,018	1,456,257	395	5,699,375	5,699,770	4,243,512 (74 %)	256,242	2,734,677	2,990,919	1,534,662 (51.4%)
256 bits	124,239	1,620,017	1,744,257	395	7,398,574	7,398,696	5,654,439 (76 %)	256,242	3,473,039	3,729,281	1,985,024 (53%)

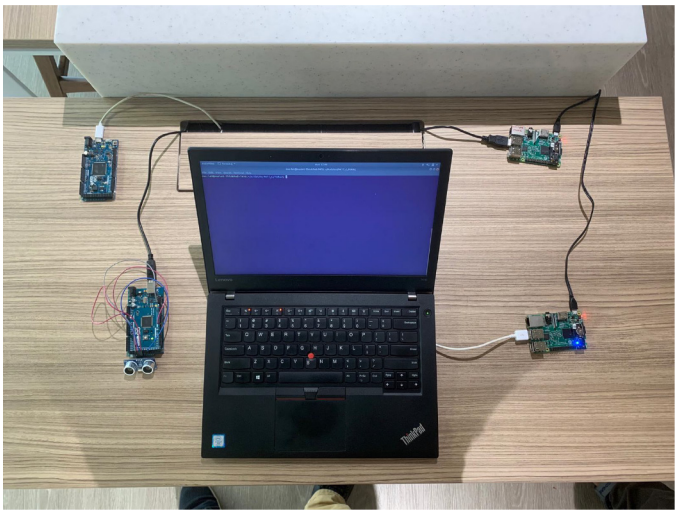


Fig. 10. Experimental setup.

are connected to the verifier (the laptop) through a wireless connection.

The latency results for HAtt are given in Table III. To compare the latency of the proposed attestation technique with the existing literature, we selected the state-of-the-art software-based attestation techniques SWATT [8] and the technique proposed in [11]. We also compare our technique with the state-of-the-art recent hybrid attestation techniques TyTan [7] and SMARM [24]. The comparison is shown in Table IV. The number of iterations and other parameters for all these techniques are selected to keep the probability of evasion for the *Adv* less than 5%. These latency results show that the proposed technique's latency is orders of magnitude lower than SWATT and SMARM. This is because SWATT calculates a hash digest of at least 50 000 addresses while SMARM calculates a hash digest over the entire memory. Moreover, we observe that the techniques proposed in [7] and [11] result in 87.5% and 15%, respectively, more latency as compared to the proposed attestation technique. This shows that the proposed attestation protocol has a very small timing footprint. Note that for HAtt, these results were obtained when the IoT device was sending its normal data to a server while for the other techniques, the IoT device was put on pause to run the attestation routine in a dedicated manner. We observe that HAtt did not affect the normal operation of the IoT device resulting in high availability.

X. PERFORMANCE ANALYSIS

In this section, we evaluate the performance of the proposed protocol using communication overhead and energy consumption. We assume a memory size of 128 kB with 128 words per block and a 64-b word size.

A. Energy Consumption

Energy is the most critical resource of IoT devices, therefore, we evaluate the energy consumption of the proposed protocol in this section. We also compare the energy consumption of our protocol with the techniques in SWATT [8], [11], TyTan [7], and SMARM [24]. These protocols were emulated on the MICA 2 mote platform using the AVRORA energy analysis tool [45]. AVRORA is a cycle-accurate instruction-level sensor network simulator with analysis tools for the Atmel AVR microcontroller and the MICA 2 sensor nodes. The MICA 2 mote uses the MPR400CB processor and a multichannel 868/910-, 433-, or 315-MHz radio transceiver.

The proposed protocol was emulated with four iterations, while for SWATT, we used the least number of iterations, i.e., 50 000. Three different key sizes, i.e., 128, 192, and 256 b, were used to compare the energy consumption as shown in Tables V and VI. We observe that the proposed protocol consumes 78%, 43.5%, 15%, and 66.4% lesser CPU energy as compared to [7], [8], [11], and [24], respectively. Similarly, the radio subsystem consumes 76%, 53%, 8.6%, and 77% lesser energy when running the proposed protocol as compared to [7], [8], [11], and [24], respectively. The total energy consumption for these protocols is shown in Fig. 11. We observe that the proposed protocol has significantly lower energy consumption.

B. Communication Overhead

The communication overhead of the proposed protocol and the protocols proposed in [7], [8], [11], and [24] is given in Table VII. We assume a key size of 128 b, a 256-b hash for authentication parameters, and a 512-b hash function for the attestation checksum.

We observe that SWATT [8] results in the lowest communication overhead. However, as described in Section X-A,

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
ENERGY CONSUMPTION FOR PROPOSED TECHNIQUE COMPARED TO EXISTING HYBRID ATTESTATION TECHNIQUES FOR A MEMORY SIZE OF 128 KB

Key Size	Proposed Technique (μJ)			TyTan [7] (μJ)			Savings (μJ)	SMARM [24] (μJ)			Savings (μJ)
	Protocol	Other	Total	Protocol	Other	Total		Protocol	Other	Total	
CPU											
128 bits	743,136	94,448	837,585	817,449	98,225	915,674	78,089 (8.5%)	1,889,090	173,544	2,062,634	1,225,049 (59.4%)
192 bits	949,646	111,815	1,061,461	1,092,092	116,287	1,208,379	146,918 (12.15%)	2,705,585	241,310	2,946,895	1,885,434 (63.9%)
256 bits	1,156,158	129,181	1,285,340	1,375,828	135,640	1,511,468	226,128 (15%)	3,519,806	308,354	3,828,160	2,542,820 (66.4%)
Radio											
128-bits	124,239	1,044,019	1,168,258	395	1,229,405	1,229,800	61,542 (5%)	808,960	2,604,712	3,413,672	2,245,414 (65.8%)
192 bits	124,239	1,332,018	1,456,257	395	1,568,544	1,568,939	112,682 (7.2%)	2,604,712	3,738,057	6,342,769	4,886,512 (76.5%)
256 bits	124,239	1,620,017	1,744,257	395	1,907,683	1,908,078	163,821 (8.6%)	2,604,712	4,852,513	7,457,225	5,712,968 (77%)

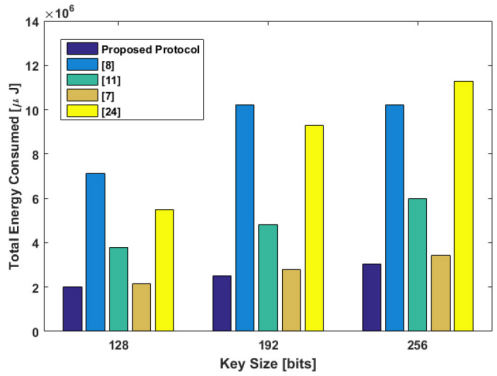


Fig. 11. Total energy consumed.

TABLE VII
COMPARISON OF COMMUNICATION OVERHEAD

Protocol	No. of Messages	No. of bytes
[8]	4	561
[11]	4	4224
[7]	4	8192
[24]	4	8192
Proposed Protocol	4	4140

SWATT results in high energy consumption. TyTan [7] and SMARM [24] result in 49.5% more communication overhead. The protocol proposed in [11] and the proposed protocol result in approximately the same communication overhead.

XI. CONCLUSION

This article proposed a protocol HAtt for the IoT devices. The proposed protocol uses PUFs to establish the hardware root of trust. Using a randomized approach, the proposed attestation technique carries out attestation on memory blocks instead of attesting the whole memory at once. This not only achieves high availability but also provides protection against the roving malware. A security analysis of the proposed attestation technique and protocol is presented to verify the corresponding security properties. The experimental validation on actual IoT devices shows that the proposed protocol has significantly lower latency and energy consumption as compared to the existing techniques.

REFERENCES

[1] F. Chen *et al.*, "An infrastructure framework for privacy protection of community medical Internet of Things—Transmission protection, storage protection and access control," *World Wide Web J.*, vol. 21, no. 1, pp. 33–57, 2018.
[2] *TPM Main Specification Level 2 Version 1.2*, Trusted Comput. Group, Beaverton, OR, USA, 2011.
[3] S. W. Smith, "Outbound authentication for programmable secure coprocessors," *Int. J. Inf. Security*, vol. 3, no. 1, pp. 28–41, 2004.
[4] M. N. Aman, "Physical unclonable functions for IoT security," in *Proc. ACM IoTPTS*, New York, NY, USA, 2016, pp. 10–13.
[5] K. Eldefrawy, G. Tsudik, A. Francillon, and D. Perito, "SMART: Secure and minimal architecture for (establishing dynamic) root of trust," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2012, pp. 1–15.
[6] P. Koeberl, S. Schulz, A.-R. Sadeghi, and V. Varadharajan, "TrustLite: A security architecture for tiny embedded devices," in *Proc. ACM Eur. Conf. Comput. Syst. (EuroSys)*, 2014, pp. 1–14.
[7] F. Brasser, "TyTAN: Tiny trust anchor for tiny devices," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2016, pp. 1–6.
[8] A. Seshadri, A. Perrig, L. van Doorn, and P. K. Khosla, "SWATT: Software-based attestation for embedded devices," in *Proc. IEEE Symp. Security Privacy*, 2004, p. 272.
[9] A. Seshadri, M. Luk, A. Perrig, L. van Doorn, and P. K. Khosla, "SCUBA: Secure code update by attestation in sensor networks," in *Proc. WiSe*, 2006, pp. 85–94.
[10] A. Seshadri, M. Luk, and A. Perrig, "SAKE: Software attestation for key establishment in sensor networks," in *Proc. Int. Conf. Distrib. Comput. Sensor Syst.*, 2008, pp. 372–385.
[11] B. Chen, X. Dong, G. Bai, S. Jauhar, and Y. Cheng, "Secure and efficient software-based attestation for industrial control devices with ARM processors," in *Proc. ACM Annu. Comput. Security Appl. Conf.*, 2017, pp. 425–436.
[12] M. N. Aman and B. Sikdar, "ATT-auth: A hybrid protocol for industrial IoT attestation with authentication," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 5119–5131, Dec. 2018.
[13] P. Tuyls and L. Batina, "RFID-tags for anti-counterfeiting," in *Topics in Cryptology CT-RSA* (LNCS 3860). Heidelberg, Germany: Springer, 2006, pp. 115–131.
[14] Y. Dodis *et al.*, "Fuzzy extractors: How to generate strong keys from biometrics and other noisy data." in *Advances in Cryptology—EUROCRYPT'2004* (Lecture Notes in Computer Science). Heidelberg, Germany: Springer-Verlag, 2004.
[15] S. Guilley and R. Pacalet, "SoCs security: A war against side-channels," *Ann. Telecommun.*, vol. 59, no. 7, pp. 998–1009, 2004.
[16] M. S. Kirkpatrick, S. Kerr, and E. Bertino, "System on chip and method for cryptography using a physically unclonable function," U.S. Patent 8 750 502 B2, Mar. 22, 2012.
[17] C. Bohm, and M. Hofer, *Physical Unclonable Functions in Theory and Practice*. New York, NY, USA: Springer, 2012.
[18] "TOTP: Time-based one-time password algorithm," IETF, RFC 6238, 2011.
[19] C. Krauß, F. Stumpf, and C. Eckert, "Detecting node compromise in hybrid wireless sensor networks using attestation techniques," in *Proc. ESAS*, 2007, pp. 203–217.
[20] H. Tan *et al.*, "A TPM-enabled remote attestation protocol (TRAP) in wireless sensor networks," in *Proc. ACM PM2HW2N*, New York, NY, USA, 2011, pp. 9–16.

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.

[21] S. Agarwal *et al.*, "Program integrity verification for detecting node capture attack in wireless sensor networks," in *Information Systems Security* (LNCS 9478), S. Jojodia and C. Manumdar, Eds. Cham, Switzerland: Springer, 2015, pp. 419–440.
[22] A. Visintin *et al. SAFE^d: Self-Attestation for Networks of Heterogeneous Embedded Devices*. Accessed: May 5, 2019. [Online]. Available: https://arxiv.org/abs/1909.08168
[23] W. Yan, A. Fu, Y. Mu, X. Zhe, S. Yu, and B. Kuang, "EAPA: Efficient attestation resilient to physical attacks for IoT devices," in *Proc. ACM IoT S&P*, 2019, pp. 2–7.
[24] X. Carpent, "Remote attestation of IoT devices via SMARM: Shuffled measurement against roving malware," in *Proc. IEEE Int. Symp. Hardw. Orient. Security Trust (HOST)*, Washington, DC, USA, 2018, pp. 9–16.
[25] C. Castelluccia, A. Francillon, D. Perito, and C. Soriente, "On the difficulty of software-based attestation of embedded devices," in *Proc. ACM Conf. Comput. Commun. Security (CCS)*, 2009, pp. 400–409.
[26] Y.-G. Choi, J. Kang, and D. Nyang, "Proactive code verification protocol in wireless sensor network," in *Proc. Int. Conf. Comput. Sci. Appl.*, vol. 2, 2009, pp. 1085–1096.
[27] J. A. Garay and L. Huelsbergen, "Software integrity protection using timed executable agents," in *Proc. ACM ASIACCS*, 2006, pp. 189–200.
[28] V. Gratzer and D. Naccache, "Alien vs. Quine," *IEEE Security Privacy*, vol. 5, no. 2, pp. 26–31, Mar./Apr. 2007.
[29] L. Gu, X. Ding, R. H. Deng, B. Xie, and H. Mei, "Remote attestation on program execution," in *Proc. ACM Workshop Scalable Trusted Comput.*, 2008, pp. 11–20.
[30] M. Jakobsson and K. A. Johansson, "Retroactive detection of malware with applications to mobile platforms," in *Proc. ACM HotSec*, 2010, pp. 1–13.
[31] M. Jakobsson and A. Juels, "Server-side detection of malware infection," in *Proc. New Security Paradigms Workshop (NSPW)*, 2009, pp. 11–22.
[32] A. Seshadri, A. Perrig, and L. van Doorn, "Using SWATT for verifying embedded systems in cars," in *Proc. Embedded Security Cars Workshop (ESCAR)*, 2004, pp. 1–4.
[33] M. Shaneck, K. Mahadevan, V. Kher, and Y. Kim, "Remote software-based attestation for wireless sensors," in *Proc. Eur. Workshop Security Ad-Hoc Sensor Netw.*, 2005, pp. 27–41.
[34] Y. Yang, X. Wang, S. Zhu, and G. Cao, "Distributed software-based attestation for node compromise detection in sensor networks," in *Proc. IEEE Int. Symp. Reliable Distrib. Syst.*, 2007, pp. 219–230.
[35] J. Kong, F. Koushanfar, P. K. Pendyala, A.-R. Sadeghi, and C. Wachsmann, "PUFatt: Embedded platform attestation based on novel processor-based PUFs," in *Proc. ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, San Francisco, CA, USA, 2014, pp. 1–6.
[36] W. Feng, Y. Qin, S. Zhao, and D. Feng, "AAoT: Lightweight attestation and authentication of low-resource things in IoT and CPS," *Comput. Netw.*, vol. 134, pp. 167–182, Apr. 2018.
[37] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. IT-29, no. 2, pp. 198–208, Mar. 1983.
[38] M. N. Aman. *ProVerif Simulation Scripts for Proposed Protocol [Software]*. Accessed: May 5, 2019. [Online]. Available: https://www.ece.nus.edu.sg/stfpage/bsikdar/scripts/HAtt
[39] B. Blanchet and B. Smyth, *ProVerif: Automatic Cryptographic Protocol Verier, User Manual and Tutorial*, INRIA, Paris, France, 2016.
[40] D. Abbasinezhad-Mood and M. Nikooghadam, "Efficient anonymous password-authenticated key exchange protocol to read isolated smart meters by utilization of extended Chebyshev chaotic maps," *IEEE Trans. Ind. Informat.*, vol. 14, no. 11, pp. 4815–4828, Nov. 2018.
[41] A. Ghani, K. Mansoor, S. Mehmood, S. A. Chaudhry, A. U. Rahman, and M. N. Saqib, "Security and key management in IoT-based wireless sensor networks: An authentication protocol using symmetric key," *Int. J. Commun. Syst.*, vol. 32, no. 16, 2019, Art. no. e4139.
[42] K. Mansoor *et al.*, "Securing IoT-based RFID systems: A robust authentication protocol using symmetric cryptography," *Sensors*, vol. 19, no. 21, p. 4752, 2019.
[43] D. Abbasinezhad-Mood, A. Ostad-Sharif, M. Nikooghadam, and S. M. Mazinani, "A secure and efficient key establishment scheme for communications of smart meters and service providers in smart grid," *IEEE Trans. Ind. Informat.*, vol. 16, no. 3, pp. 1495–1502, Mar. 2020.
[44] R. Canetti and H. Krawczyk, "Universally composable notions of key exchange and secure channels," in *Proc. EUROCRYPT*, Amsterdam, The Netherlands, 2002, pp. 337–351.
[45] B. L. Titzer, D. K. Lee, and J. Palsberg, "Avrora: Scalable sensor network simulation with precise timing," in *Proc. IEEE IPSN*, 2005, pp. 477–482.



Muhammad Naveed Aman received the B.Sc. degree in computer systems engineering from the University of Engineering and Technology, Peshawar, Pakistan, in 2006, the M.Sc. degree in computer engineering from the Center for Advanced Studies in Engineering, Islamabad, Pakistan, in 2008, and the M.Eng. degree in industrial and management engineering and the Ph.D. degree in electrical engineering from Rensselaer Polytechnic Institute, Troy, NY, USA, in 2012.

He served as an Assistant Professor with the Faculty of National University of Computer and Emerging Sciences, Islamabad. He is currently working as a Senior Research Fellow with the Department of Computer Science, National University of Singapore, Singapore. His research interests include IoT and network security, wireless and mobile networks, and secure embedded systems.



Mohamed Haroon Basheer (Student Member, IEEE) received the B.Tech. degree in electronic engineering from the National University of Singapore, Singapore, where he is currently pursuing the Master of Computing degree in computer science.

Since November 2017, he has been a Research Assistant with the NUS-Singtel Cyber Security Research and Development Laboratory, National University of Singapore.



Siddhant Dash received the B.Tech. degree in electronics and communication engineering from the International Institute of Information Technology, Naya Raipur, India, in 2019. He is currently pursuing the master's degree in engineering management with Cornell University, Ithaca, NY, USA.

He contributed to this research work during his final semester of undergraduate studies when he worked as a Research Intern with the NUS-Singtel Cyber Security Research and Development Laboratory, National University of Singapore, Singapore.



Jun Wen Wong received the bachelor's degree in electrical engineering from Nanyang Technology University, Singapore, in 2007, and the master's degree in computer engineering from the National University of Singapore, Singapore, in 2010.

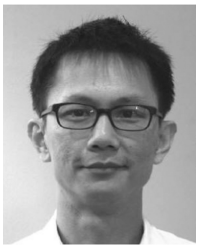
He has more than ten years of work experience in cybersecurity field. He was a Senior Research Engineer with the Institute for Infocomm Research, Agency for Science, Technology and Research, Singapore. He is currently a Senior Research and Development Engineer and one of the Co-Principal Investigators with Singtel Cybersecurity Research and Development in Trustwave, Singtel Company, Singapore. He was involved in the European Union and Singapore Government funded projects relating to critical infrastructures, such as sensor network, energy, and rail transport.

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.



Jia Xu received the Ph.D. degree in computer science from the National University of Singapore, Singapore, in 2012.

He worked as a Research Scientist with the Institute for Infocomm Research, Singapore, from 2012 to 2017. He is a Cyber Security Research and Development Manager specializing in cloud and data security and post-quantum security with the NUS-SingTel Cyber Security Research and Development Laboratory, National University of Singapore. His research interests include applied cryptography and cloud computing security.



Hoon Wei Lim received the Ph.D. degree in information security from the Royal Holloway, University of London, Egham, U.K.

He held research positions with the Institute for Infocomm Research, Singapore, the National University of Singapore, Singapore, Nanyang Technological University, Singapore, and SAP, Levallois-Perret, France. He is currently an Associate Director with the NUS-Singtel Cyber Security Research and Development Laboratory, National University of Singapore. His recent research interests have been centered around data security and privacy, security intelligence and analytics within enterprize environments, and cyber–physical systems.



Biplab Sikdar received the B.Tech. degree in electronics and communication engineering from North Eastern Hill University, Shillong, India, in 1996, the M.Tech. degree in electrical engineering from the Indian Institute of Technology Kanpur, Kanpur, India, in 1998, and the Ph.D. degree in electrical engineering from Rensselaer Polytechnic Institute, Troy, NY, USA, in 2001.

He was on the faculty of Rensselaer Polytechnic Institute from 2001 to 2013, first as an Assistant Professor and then as an Associate Professor. He is currently an Associate Professor with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore. His research interests include wireless network, security for IoT, and cyber–physical systems.

Dr. Sikdar served as an Associate Editor for the IEEE TRANSACTIONS ON COMMUNICATIONS from 2007 to 2012. He currently serves as an Associate Editor for the IEEE TRANSACTIONS ON MOBILE COMPUTING. He is a member of Eta Kappa Nu and Tau Beta Pi.

Authorized licensed use limited to: Universita degli Studi di Verona. Downloaded on June 03,2025 at 12:46:09 UTC from IEEE Xplore. Restrictions apply.